Share ···

What is the optimal Hermes skill token/character size?

I'll pin this to the practical "agent skill file" use case, not generic LLM context theory. I'm going to sanity-check whether Hermes/OpenClaw documents publish a hard limit, then give you the working sizing rule.

Thought for 12s ›

For **Hermes skills**, I'd target:

Optimal `SKILL.md` size:
2,000–6,000 tokens
roughly **8,000–24,000 characters**

Hard practical ceiling:
~10,000 tokens / **~40,000 characters** before it starts getting chunky.

Best shape:

```
SKILL.md
  1. Purpose / when to use
  2. Required assumptions
  3. Step-by-step procedure
  4. Commands / templates
  5. Failure modes
  6. Links to optional reference files
```

Hermes uses progressive disclosure: it loads the compact skills list first, then only loads the full skill content when needed. Official docs say the skill list is around **3k tokens**, and `skill_view(name)` loads the full skill only on demand. Hermes Agent

So the move is:

```
Keep SKILL.md lean.
Put big docs in separate reference files.
```

Example:

```
~/.hermes/skills/kubernetes-expert/
  SKILL.md              # 3k–6k tokens
  references/
    troubleshooting.md  # big
    helm-patterns.md
```

```
security-baseline.md
k3s-homelab.md
```

For your Hermes/Kubernetes-type skills, I'd use this rule:

File	Target size
`SKILL.md`	**10k–20k chars**
Heavy reference docs	**20k–60k chars each**
SOUL.md	**5k–15k chars max**
USER.md / memory facts	compact as hell

Hermes itself requires at least a **64k token context model**, but that does **not** mean you should write 64k-token skills. That just gives the agent working room for tools, repo context, chat history, and loaded skill content. Hermes Ag…

My no-BS recommendation:

Aim for 12,000–18,000 characters per main skill.
That is big enough to be useful, small enough that Hermes won't drag ass or waste tokens every time it loads it.

Lets say I have 4+ subskills in mind that hermes could selectively choose from as designated by a main skill instruction. Break down the optimal and maximal character limits

Thought for 4s ›

Yeah — for **one main Hermes skill routing to 4+ subskills**, you want the **main skill tiny and opinionated**, with each subskill only loaded when needed.

Hermes skills use progressive disclosure: the agent first sees a compact skill list, then can load `SKILL.md`, then can load specific files inside that skill directory when needed. That means your main file should act like a **router**, not the whole brain. Hermes Agent

Recommended layout

```
~/.hermes/skills/kubernetes-expert/
  SKILL.md
```

```
  subskills/
    cluster-troubleshooting.md
    helm-authoring.md
    network-ingress.md
    storage-ceph-nfs.md
    security-hardening.md
  references/
    command-cookbook.md
    known-errors.md
    manifests/
```

Character/token sizing

Approximate conversion:

```
1 token ≈ 4 characters
```

Not exact, but good enough for sizing.

Piece	Optimal chars	Optimal tokens	Max chars	Max tokens
Main `SKILL.md` router	**4k–10k**	**1k–2.5k**	**16k**	**4k**
Each subskill	**8k–20k**	**2k–5k**	**32k–40k**	**8k–10k**
Reference file	**10k–30k**	**2.5k–7.5k**	**60k**	**15k**
Whole skill package	**40k–120k**	**10k–30k**	**200k–300k**	**50k–75k**

Best practical target for 4 subskills

For your use case, I'd do:

```
SKILL.md router:        6k–8k chars
Subskill 1:             12k–18k chars
Subskill 2:             12k–18k chars
Subskill 3:             12k–18k chars
Subskill 4:             12k–18k chars
Shared references:      20k–60k chars total
```

Total package:

```
~80k–140k chars total
~20k–35k tokens total
```

But runtime load should usually be only:

```
Router + 1 subskill
≈ 18k–26k chars
≈ 4.5k–6.5k tokens
```

That's the sweet spot.

Main `SKILL.md` should not contain deep content

The main skill should mostly be a routing table:

Markdown

```
# Kubernetes Expert

Use this skill for Kubernetes, K3s, Helm, ingress, storage, networking, security, and clu

## Routing

### Use `subskills/cluster-troubleshooting.md` when:
- Pods are Pending, CrashLoopBackOff, ImagePullBackOff, or stuck Terminating
- Nodes are NotReady
- DNS, kubelet, containerd, or CNI behavior is suspicious
- The user pasted `kubectl describe`, logs, or events

### Use `subskills/network-ingress.md` when:
- The task involves Traefik, IngressRoute, Gateway API, cert-manager, ExternalDNS, Cloudf

### Use `subskills/storage-ceph-nfs.md` when:
- The task involves PVCs, PVs, StorageClasses, NFS, Rook/Ceph, Longhorn, CSI, permissions

### Use `subskills/helm-authoring.md` when:
- The task involves creating, patching, upgrading, or debugging Helm charts

### Use `subskills/security-hardening.md` when:
- The task involves RBAC, NetworkPolicy, Pod Security Standards, secrets, admission contr
```

Then each subskill should be procedural:

Markdown

```
# Storage / Ceph / NFS Subskill

## Use when
...

## First commands to run
...

## Diagnosis flow
...
```

```
## Known failure modes
...

## Safe fixes
...

## Escalation path
...
```

Max limits I would not cross

For Hermes specifically, I would avoid:

```
Main SKILL.md above 16k chars
Any subskill above 40k chars
Any single reference above 60k chars
Package above 300k chars unless heavily indexed
```

Can it technically work bigger? Probably. Should you? No. Large raw skill bodies create token drag and attention dilution; recent skill-optimization research found skill bodies often contain a ton of non-actionable content, and restructuring into progressive disclosure can cut body size substantially while improving quality. arXiv

My recommended “Z-tier” pattern

For a serious Hermes engineering skill:

```
Main router:
  6k–8k chars

Each operational subskill:
  12k–18k chars

Each big reference:
  20k–40k chars

Each command/template file:
  small, focused, copy-pasteable
```

Do **not** make one mega `SKILL.md`. Make a **thin router + focused subskills + optional references**. That keeps Hermes sharp instead of shoving a wiki into its face every time.

Rephrase based off this

```
starcitizen-scmdb/
├── SKILL.md                    # skill entry point & routing
├── DesignDoc.md
├── README.md
├── Star Citizen APIs.pdf
│
├── data/                       # versioned cache (committed to
repo)
│   ├── VERSION                 # "4.1.1-live.9581966"
```

Show more

Thought for a second

Based on **your actual repo layout**, I'd size it like this:

```
starcitizen-scmdb/
├── SKILL.md              # tiny router, not the knowledge dump
├── DesignDoc.md          # architecture + intent
├── README.md             # human-facing usage
├── Star Citizen APIs.pdf # offline reference only
├── data/                 # versioned facts/cache
├── schemas/              # contracts
├── scripts/              # executable update/fetch logic
└── instructions/         # selective subskills
```

Optimal character limits

File / folder	Role	Optimal size	Max before it gets dumb
`SKILL.md`	Hermes entry point + router	**4k–8k chars**	**12k–16k chars**
`instructions/missions.md`	Mission query/use instructions	**8k–16k chars**	**30k–40k chars**
`instructions/fabricator.md`	Fabricator/material crafting logic	**8k–18k chars**	**35k–45k chars**
`instructions/resources.md`	Resources/prices/locations logic	**8k–16k chars**	**30k–40k chars**

File / folder	Role	Optimal size	Max before it gets dumb
`instructions/mining_solver.md`	Mining loadout/solver reasoning	**12k–24k chars**	**45k–60k chars**
`DesignDoc.md`	System design and rationale	**10k–25k chars**	**50k–75k chars**
`README.md`	Human repo usage	**4k–12k chars**	**20k–30k chars**
`schemas/*.json`	Machine contracts	As needed	Keep precise
`data/*.json`	Versioned cache	As needed	Do not manually summarize into skill
`scripts/*.py`	Fetch/scrape behavior	As needed	Split if huge
`Star Citizen APIs.pdf`	Reference	Any size	Should not be auto-loaded

Best target for this repo

For this specific `starcitizen-scmdb` skill, I'd aim for:

```
SKILL.md:                       6k–8k chars

instructions/missions.md:       10k–14k chars
instructions/fabricator.md:     12k–18k chars
instructions/resources.md:      10k–14k chars
instructions/mining_solver.md:  16k–24k chars

DesignDoc.md:                   15k–25k chars
README.md:                      6k–10k chars
```

That gives you a total "AI instruction surface" around:

```
~60k–100k characters
~15k–25k tokens
```

But Hermes should usually load only:

```
SKILL.md + 1 instruction file + relevant schema/data snippet
```

Runtime working load should usually stay around:

```
~16k–35k characters
~4k–9k tokens
```

That's the actual sweet spot.

How `SKILL.md` should behave

`SKILL.md` should **not** explain all Star Citizen APIs, mining math, mission schemas, and fabricator details inline.

It should route.

Example structure:

Markdown

```
# Star Citizen SCMDB Skill

Use this skill when the user asks about Star Citizen structured data, SCMDB, missions, re

## Data version

Before answering from cached data:
1. Read `data/VERSION`.
2. Prefer matching files under `data/<module>/`.
3. Tell the user which game/cache version the answer uses.
4. If the requested answer depends on volatile prices, locations, or patch data, recommen

## Routing

### Missions
Load `instructions/missions.md` when the user asks about:
- Mission rewards
- Mission types
- Reputation-gated missions
- Location-based missions
- Mission comparison
- Mission JSON structure

Relevant files:
- `data/missions/4.1.1-live.json`
- `schemas/missions.schema.json`

### Fabricator
Load `instructions/fabricator.md` when the user asks about:
- Crafting
- Fabrication recipes
- Required materials
- Build chains
- Crafting cost comparison
```

```
Relevant files:
- `data/fabricator/4.1.1-live.json`
- `schemas/fabricator.schema.json`

### Resources
Load `instructions/resources.md` when the user asks about:
- Commodities
- Materials
- Resource locations
- Sell/buy prices
- Refining inputs/outputs
- UEX or SCMDB resource data

Relevant files:
- `data/resources/4.1.1-live.json`
- `schemas/resources.schema.json`

### Mining solver
Load `instructions/mining_solver.md` when the user asks about:
- Mining heads
- Modules
- Ship mining setups
- Rock breakability
- Instability/resistance
- Optimal loadouts
- Mining yield calculations

Relevant files:
- `data/mining/equipment.json`
- `data/mining/4.1.1-live.json`
- `schemas/mining.schema.json`
```

Character budget by responsibility

`SKILL.md`

Keep it tight:

```
Purpose:                500–1,000 chars
Version/cache rules:    800–1,500 chars
Routing table:          2,000–4,000 chars
Safety/accuracy rules:  1,000–2,000 chars
Output expectations:    500–1,000 chars
```

Total:

```
~5k–9k chars
```

Do **not** put API docs, schema details, or solver math in here.

`instructions/missions.md`

Good size:

```
10k–14k chars
```

Should include:

```
- When to use this subskill
- Which files to inspect
- Mission lookup flow
- How to compare missions
- How to handle stale patch data
- Output formats
- Known gotchas
```

Max:

```
~35k chars
```

Past that, split into:

```
instructions/missions.md
instructions/mission-reputation.md
instructions/mission-rewards.md
```

`instructions/fabricator.md`

Good size:

```
12k–18k chars
```

This one can be slightly bigger because crafting logic tends to branch.

Should include:

```
- Recipe lookup flow
- Material dependency tracing
- Cost/yield reasoning
- Missing-data behavior
- Fabricator output templates
```

Max:

```
~45k chars
```

`instructions/resources.md`

Good size:

```
10k–14k chars
```

Should include:

```
- Resource lookup rules
- SCMDB vs UEX vs Wiki priority
- Patch/version handling
- Buy/sell price caveats
- Location confidence rules
```

Max:

```
~40k chars
```

`instructions/mining_solver.md`

Good size:

```
16k–24k chars
```

This is your heaviest subskill.

It probably needs room for:

```
- Equipment lookup
- Rock stat interpretation
- Mining head/module selection
- Breakability checks
- Yield/value calculations
- Failure cases
- “Don’t hallucinate missing constants” rules
```

Max:

```
~60k chars
```

If it goes past that, split it:

```
instructions/mining_solver.md
instructions/mining_equipment.md
instructions/mining_math.md
instructions/mining_output_formats.md
```

Max package size

For the full repo, I'd separate **instruction size** from **data size**.

AI instruction package

```
SKILL.md + instructions/*.md + DesignDoc.md
```

Optimal:

```
60k–120k chars
15k–30k tokens
```

Maximum before it becomes bloated:

```
200k–250k chars
50k–62k tokens
```

Full repo including data/cache/scripts/PDF

No real character limit because Hermes should not blindly load all of it.

But practically:

```
data/      can be large
schemas/   should be compact
scripts/   can be normal source-code size
PDF        reference only
```

The important rule:

```
Never copy giant data dumps into SKILL.md or instructions/*.md.
Point Hermes to the data files instead.
```

Best final recommendation

For `starcitizen-scmdb`, I'd use this sizing model:

```
SKILL.md
  6k–8k chars
  Router only.

instructions/missions.md
  10k–14k chars

instructions/fabricator.md
  12k–18k chars

instructions/resources.md
  10k–14k chars

instructions/mining_solver.md
  16k–24k chars

DesignDoc.md
  15k–25k chars

README.md
  6k–10k chars
```

And I would cap the important files like this:

```
SKILL.md:                      16k chars absolute max
Each normal instruction file:  40k chars max
Mining solver:                 60k chars max
DesignDoc.md:                  75k chars max
Single reference doc:          75k chars max
```

The clean design is:

```
SKILL.md tells Hermes where to look.
instructions/*.md tells Hermes how to reason.
schemas/*.json tells Hermes what shape is valid.
data/*.json tells Hermes what is true for the current patch.
scripts/*.py tells Hermes how to refresh reality.
```

That structure is solid. Don’t turn `SKILL.md` into a Star Citizen encyclopedia. Let it be the dispatcher.